

EE5203 L10 Study Sheet

UART and PC communication - Basri Erdogan

Essential question: Two devices share one wire and no clock. How do they agree on where each bit begins — and what happens when they disagree?

What you should be able to do

- Draw an 8N1 frame and count its bit times.
- Convert between baud, microseconds per byte, and lines per second.
- Compute BRR from PCLK and a baud rate, and the resulting error.
- Transmit formatted telemetry from the tick without blocking the loop.
- Receive by interrupt, re-arm correctly, and act in `main()`.

The frame

```
idle  START  D0 D1 D2 D3 D4 D5 D6 D7  STOP  idle
high  low    <---- LSB first ----->  high
      |<----- 10 bit times ----->|
```

8N1 = 8 data bits, No parity, 1 stop bit. One byte therefore costs **10 bit times** — the framing overhead is 20 %.

$$\text{time per byte} = \frac{10}{\text{baud}} \quad \text{bytes per second} = \frac{\text{baud}}{10}$$

Baud	µs/byte	bytes/s	18-char line
9 600	1041.67	960	18.75 ms
57 600	173.61	5 760	3.13 ms
115 200	86.81	11 520	1.56 ms

There is no clock wire. Both ends agree the rate in advance; the receiver re-synchronises on each start bit and then samples in the middle of each bit time. Agreement on baud, data bits, parity, and stop bits must be exact — a UART cannot report a disagreement, only hand you its consequences.

Vocabulary table

Name	Job
Baud	bits per second on the wire , including framing
BRR	the divider that makes the baud from PCLK
TXE	transmit register empty — write the <i>next</i> byte
TC	transmission complete — the line is genuinely idle
RXNE	a byte has arrived; reading DR clears it
ORE	overrun — a byte arrived before you read the last one; it is lost
VCP	virtual COM port — the ST-LINK's USB-to-serial bridge

Computing BRR

USART2 is on **APB1**; at the default HSI, PCLK1 = 16 MHz. Oversampling by 16:

$$\text{USARTDIV} = \frac{f_{\text{PCLK}}}{16 \times \text{baud}}$$

BRR stores this as fixed point: **top 12 bits = integer part, bottom 4 bits = sixteenths.**

For 115 200: $16\,000\,000 / (16 \times 115\,200) = 8.6806 \rightarrow$ mantissa **8**, fraction $0.6806 \times 16 = 11 \rightarrow \text{BRR} = (8 \ll 4) | 11 = \mathbf{139} = 0x08B \rightarrow \text{actual} = 16\,000\,000 / (16 \times 8.6875) = \mathbf{115\,108\,baud}$, error **−0.08 %**

Baud	BRR	Actual	Error
9 600	0x683	9 598	−0.02 %
38 400	0x1A1	38 369	−0.08 %
115 200	0x08B	115 108	−0.08 %
230 400	0x045	231 884	+0.64 %

A UART tolerates roughly ± 2 % total across both ends. Small errors are invisible; 0.64 % consumes a third of the budget before the other device has contributed anything.

The board already has the wire

PA2 (USART2_TX) and PA3 (USART2_RX) are wired to the ST-LINK, which presents a serial port over the same USB cable that flashes the board. No dongle, no level shifter. **Close the Serial Monitor before flashing** — an open port blocks the programmer.

Transmitting

```
char buf[48];
int n = snprintf(buf, sizeof buf, "%lu,%u\r\n",
                (unsigned long)g_ms, (unsigned)raw);
HAL_UART_Transmit(&huart2, (uint8_t *)buf, (uint16_t)n, 100);
```

- **snprintf, never sprintf.** snprintf truncates at the buffer size; sprintf writes past the end and corrupts whatever is next on the stack.
- **HAL_UART_Transmit blocks** until the last byte is handed over — 1.56 ms for an 18-character line at 115 200. Acceptable in main() at 10 Hz; never acceptable in a callback.
- **\r\n**, not **\n** alone — most monitors need both to return to column 1.
- Comma-separated with a timestamp pastes straight into a spreadsheet.

TXE versus TC. TXE means “the holding register is free, give me the next byte” — the previous byte may still be shifting out. TC means the shift register is empty too and the line is idle. Powering down, or turning a half-duplex transceiver around, needs TC.

Receiving

```
HAL_UART_Receive_IT(&huart2, &rx_byte, 1); /* arm once */

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2) {
        rx_ring[head] = rx_byte;
        head = (head + 1) % RX_SIZE;
        HAL_UART_Receive_IT(&huart2, &rx_byte, 1); /* RE-ARM - mandatory */
    }
}
```

- **Never HAL_UART_Receive blocking.** It waits for a human; meanwhile nothing else in the system runs.
- **The re-arm is mandatory.** HAL disarms after each completed receive. Omit it and you receive exactly one byte, ever — a silence that looks like broken hardware.
- The callback **records**; main() **decides**. Same discipline as every interrupt since L06.
- Reading DR clears RXNE — read-to-clear, like the ADC’s EOC.

ORE. Set when a byte arrives before the previous one was read. That byte is gone, and HAL delivers nothing further until the error is cleared. Usual causes: blocking somewhere long, a missing re-arm, or a breakpoint held while data arrives.

SFR evidence

Claim	Evidence
the baud divider is right	USART2->BRR = 0x08B for 115 200 at 16 MHz
the pins belong to the USART	MODER = 10 for PA2/PA3; AFR[0] selects AF7
a byte is waiting	USART2->SR RXNE = 1; it clears when DR is read

Claim	Evidence
bytes were lost the line is idle	USART2->SR ORE = 1 after a held breakpoint TC = 1 after the last byte

Common mistakes

- Baud mismatch between board and monitor — consistent garbage, no warning.
- Receive interrupt never re-armed — one byte, then permanent silence.
- `sprintf` into a small buffer — silent corruption.
- `HAL_UART_Transmit` inside a callback — milliseconds of blocking in interrupt context.
- ORE never cleared — reception dead for good.
- `\n` without `\r` — the display stair-steps.
- Serial Monitor left open while flashing — the programmer cannot claim the port.
- `printf` with floats for telemetry — huge, slow, one transmit per character.

Mastery self-check

- ☐ I can draw an 8N1 frame and say why a byte costs 10 bit times.
- ☐ I can compute $\mu\text{s}/\text{byte}$ and max lines/s for any baud.
- ☐ I can compute BRR and the resulting baud error.
- ☐ I can state the $\pm 2\%$ tolerance and why it matters.
- ☐ I can distinguish TXE from TC, and RXNE from ORE.
- ☐ I can write an interrupt receive with a correct re-arm.
- ☐ I can diagnose garbage, silence, and one-byte-then-nothing by symptom.

Five review questions

1. At 57 600 baud, 8N1, how long does a 17-character line take? What fraction of a 50 ms budget is that, and how many such lines per second could the link carry flat out?
2. Compute BRR for 19 200 baud with $PCLK1 = 16\text{ MHz}$. Give mantissa, fraction, hex value, actual baud, and error.
3. A student's board sends its banner correctly, but every character the *PC* sends is ignored after the first one. Name the single missing line.
4. Explain why `HAL_UART_Transmit(&huart2, buf, 18, 100)` is acceptable in `main()` at 10 Hz but not inside `HAL_TIM_PeriodElapsedCallback` on a 1 ms tick. Use numbers.
5. Your monitor shows readable text that occasionally drops characters, and ORE is set. Give two distinct causes and the fix for each.

See the L10 worksheet for extended practice. Worked solutions are released after the practice window.